

AI-Driven Personalized News Recommendation System

Business Requirement Document

CONTENTS

1. EXECUTIVE SUMMARY	1
2. PROJECT VISION	1
3. PROJECT MISSION	2
4. PROJECT OBJECTIVES.....	2
5. CURRENT SYSTEM (AS IS STATE)	2
6. CHALLENGES IN CURRENT SYSTEM.....	3
7. PROPOSED SOLUTION (HIGH-LEVEL).....	3
8. MIND MAP	4
9. COMPETITOR ANALYSIS.....	4
10. SWOT ANALYSIS	5
11. PROJECT SCOPE	6
11.1 IN SCOPE (PHASE 1).....	6
11.2 OUT OF SCOPE.....	6
11.3 EXCLUSIONS.....	6
12. STAKEHOLDER MATRIX	7
13. FUNCTIONAL REQUIREMENTS	7
13.1 NEWS COLLECTION & PROCESSING REQUIREMENTS.....	7
13.2 USER ACCOUNT & PERSONALIZATION REQUIREMENTS.....	8
13.3 AI VECTORIZATION REQUIREMENTS	8
13.4 RECOMMENDATION LOGIC REQUIREMENTS.....	9
13.5 SEMANTIC SEARCH REQUIREMENTS.....	9
13.6 USER INTERFACE REQUIREMENTS.....	9
14. NON-FUNCTIONAL REQUIREMENTS	10
NFR1 - PERFORMANCE	10
NFR2 – SCALABILITY.....	10
NFR3 - SECURITY	10
NFR4 - RELIABILITY	10

NFR5 - USABILITY.....	10
NFR6 - MAINTAINABILITY.....	11
15. RISKS & MITIGATION.....	11
16. MODULE WISE ALLOCATION.....	11
17. HIGH LEVEL DESIGN	11
18. LOW LEVEL DESIGN	12
19. PROPOSED TECH STACK.....	16
20. PROPOSED ARCHITECTURE	17
21. DELIVERABLES AND MILESTONES	17
22. SUCCESS CRITERIA.....	18
23. ASSUMPTIONS & CONSTRAINTS	18
23.1 - ASSUMPTIONS.....	18
23.2 - CONSTRAINTS.....	18

AI-Driven Personalized News Recommendation System

Business Requirements Document (BRD)

1. Executive Summary

Users today are overwhelmed by the sheer volume of online news, making it difficult to find content aligned with their interests. Traditional news platforms provide static, non-personalized feeds that fail to engage users effectively. This project aims to build an AI-driven personalized news recommendation system that leverages user behaviour, preferences, and semantic understanding of news content to deliver highly relevant, continuously updated news feeds. By integrating advanced vectorization models, relevance scoring, and semantic search capabilities, the platform intends to improve user engagement, satisfaction, and retention across digital channels.

2. Project Vision

- To develop an intelligent and adaptive news recommendation platform that delivers **context-aware and personalized news** content using artificial intelligence.
- To address information overload by enabling users to consume **relevant, meaningful, and interest-aligned news** efficiently.
- To leverage semantic understanding and machine learning techniques to enhance content discovery beyond traditional category-based systems.
- To establish a scalable and modular foundation for future advancements in personalized digital media delivery.

3. Project Mission

- To design and implement an AI-driven personalized news recommendation system utilizing semantic vector embeddings and similarity-based relevance modeling.
- To continuously refine user profiles by analyzing interaction patterns and dynamically updating user interest representations.
- To provide an intuitive, responsive, and user-friendly interface that enhances engagement and ease of navigation.
- To employ open-source and cost-effective technologies while ensuring system performance, reliability, and data security.

4. Project Objectives

- Deliver personalized news recommendations in real time based on user interests and reading behaviour.
- Aggregate, classify, and process news from diverse and credible sources.
- Enhance user engagement through tailored content discovery and semantic search.
- Reduce user effort in finding relevant news by automating content curation using AI.
- Provide a smooth, intuitive user interface accessible across devices.

5. Current System (As Is state)

1. Users primarily consume news through **general news websites and applications** that aggregate content from multiple sources.
2. News content is typically displayed in **static or semi-static feeds**, based on popularity, recency, or broad categories such as politics, sports, or technology.
3. User preferences are captured, if at all, through **manual category selection or subscription to topics**, with limited adaptation over time.
4. Search functionality in existing systems is **largely keyword-based**, matching exact words in headlines or summaries.

5. News recommendations do not significantly adapt based on **individual reading behavior**, such as articles clicked or time spent reading.
6. Content relevance is generally determined by **editorial rules or predefined ranking logic**, rather than semantic analysis of user interests.
7. Users must manually browse or scroll through large volumes of articles to find content of interest.

6. Challenges in current system

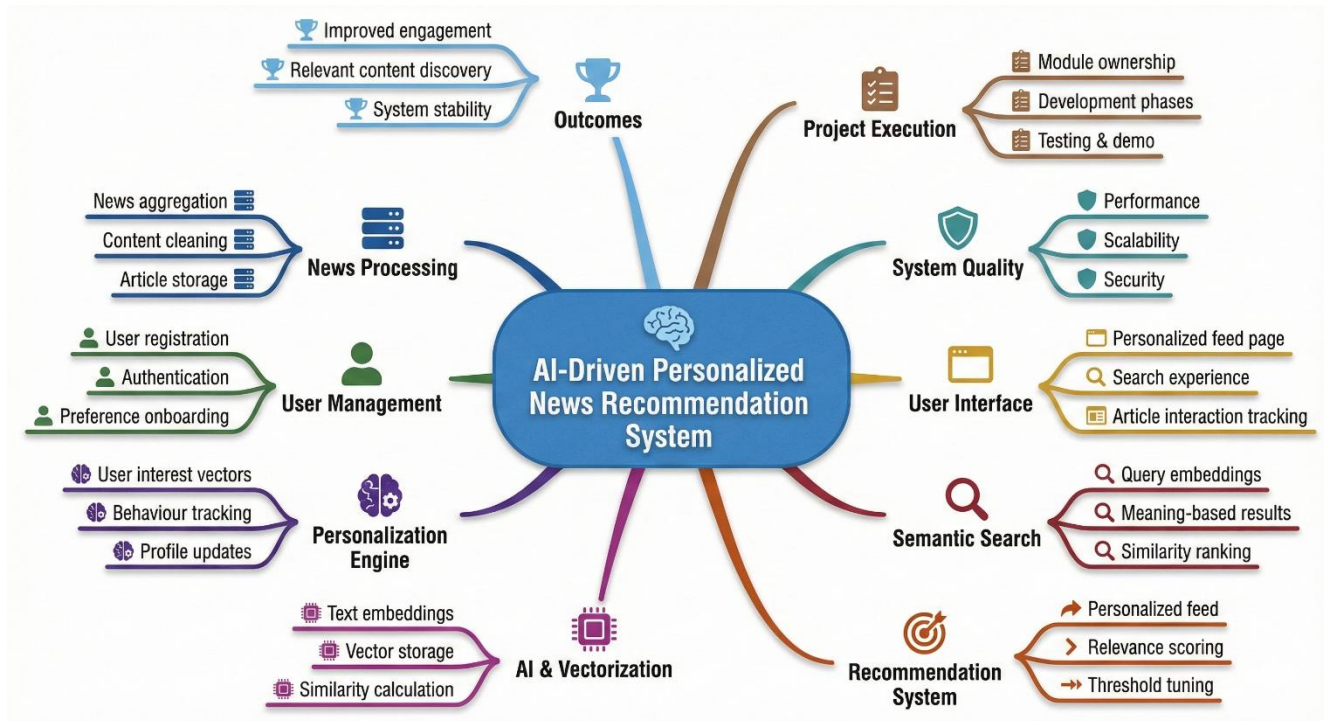
- Information Overload – Users struggle to filter relevant news from vast online content.
- Generic News Feeds – Existing platforms often push trending stories rather than personalized content.
- No Understanding of User Intent – Traditional systems rely on categories/tags rather than semantic meaning.
- Poor Search Quality – Keyword-based search returns irrelevant or incomplete results.
- Static Personalization – User interests change over time but existing systems rarely adapt dynamically.

7. Proposed Solution (High-Level)

The project proposes an AI-powered personalized news recommendation system that:

- Aggregates real-time news and preprocesses content.
- Generates semantic vector embeddings for both articles and users using.
- Calculates relevance, ensuring personalized recommendations.
- Supports AI-driven semantic search that understands query meaning, not just keywords.
- Evolves user profiles by updating preference vectors based on reading history.
- Provides a clean, intuitive web interface for reading and searching.
- This solution improves engagement by ensuring that every user's feed is truly personalized and dynamically updated.

8. Mind Map



9. Competitor Analysis

Google News operates as a large-scale news aggregation platform using advanced machine-learning models for personalization based on extensive user and publisher signals. Its strengths include massive content coverage, strong ML infrastructure, and deep publisher integration.

Apple News is a privacy-focused, curated news platform tightly integrated with the iOS ecosystem. Personalization combines editorial selection with on-device machine-learning while prioritizing user privacy. Although it offers a strong user experience and subscription integration, its platform lock-in create opportunities for cross-platform alternatives.

Flipboard provides a magazine-style news experience centered on topic selection and editorial curation, supported by adaptive learning from user reading behavior. It excels in user experience and content discovery but relies less on deep behavioral machine-learning.

Feedly targets professionals and researchers, offering user-defined feeds enhanced with AI for summarization, trend detection, and prioritization. Its personalization focuses on efficiency and analysis rather than mass-market consumption.

SmartNews emphasizes trending and local news discovery using a combination of algorithmic ranking and human curation. Personalization is driven primarily by freshness, popularity, and regional relevance. While effective for quick discovery, it lacks deep long-term behavioral personalization.

Microsoft Start (MSN) functions as a news aggregation service integrated with Microsoft products such as Windows and Bing. It offers basic personalization through customizable feeds and widgets.

10. SWOT Analysis

Strengths

- AI-driven personalization improves content relevance and user engagement.
- Adaptive learning from user behavior over time.
- Scalable architecture suitable for large user bases.
- Ability to provide explainable and transparent recommendations.

Weaknesses

- Cold-start problem for new users and new articles
- High dependency on quality and volume of user data
- Increased system complexity and infrastructure cost
- Potential user concerns regarding data privacy

Opportunities

- Growing demand for personalized digital content
- Potential partnerships with news publishers
- Expansion into regional and multilingual news markets
- Monetization through premium personalization features and targeted advertising.

Threats

- Strong competition from established tech companies
- Privacy regulations limiting data collection

- Risk of algorithmic bias and filter bubbles
- Spread of misinformation affecting platform trust

11. Project Scope

11.1 In Scope (Phase 1)

1. News aggregation from various APIs and RSS feeds.
2. User profile management, behaviour tracking, and preference storage.
3. AI-based vectorization and embedding generation for articles and users.
4. Relevance-based recommendation engine using vector similarity.
5. Semantic search for relevant news.
6. User interface for personalized feeds and search results.

11.2 Out of Scope

1. Creation of original news content.
2. Verifying legitimacy of news content.
3. Social media features such as commenting and sharing.
4. Ranking the news by relevance.

11.3 Exclusions

- **User Behaviour Interpretation Beyond System Interaction:** The system does not interpret user intent outside recorded actions such as article clicks or reads.
- **Manual Content Curation or Human Intervention:** No human involvement is included in filtering, recommending, or moderating news content.
- **Guarantee of Recommendation Accuracy:** The system does not guarantee correctness, completeness, or absolute relevance of recommended news.
- **Real-Time or Live News Guarantees:** The system does not ensure real-time delivery of breaking news or instant feed updates from sources.
- **Long-Term Data Retention Policies:** Archival, backup, and long-term retention of historical news or user data are not addressed.

12. Stakeholder Matrix

Stakeholder	Role / Interest
End Users	Consume personalized news and interact with the system. Expect relevant and curated content.
Development Team	Responsible for design, development, testing, and deployment.
UI/UX Designers	Creates intuitive and responsive user interfaces.
Data Engineer	Manages data ingestion, cleaning, and storage.
AI/ML Developers	Implements vector embeddings and semantic relevance logic.

13. Functional Requirements

13.1 News Collection & Processing Requirements

FR-1: Multi-source News Aggregation

The system must automatically fetch news articles from multiple trusted RSS providers (e.g., BBC, TechCrunch) at regular intervals.

- Should support easy addition/removal of sources.
- Must handle varied feed formats and ensure consistency in extracted fields (title, summary, link, timestamp).

FR-2: Content Cleaning & Standardization

Fetches news summaries must be cleaned by removing HTML tags, special characters, and formatting inconsistencies.

- Cleaned text should retain original meaning.
- Must ensure data quality before sending to vectorization.

FR-3: Article Storage & Indexing

All cleaned news articles must be stored with metadata.

- Must also store vector embeddings (from Module 3).
- Should allow efficient querying by timestamp, category, and similarity search.

13.2 User Account & Personalization Requirements

FR-4: Secure User Registration & Authentication

System must allow users to create accounts and log in securely.

- Passwords must be salted + hashed.
- Sessions must be maintained securely.

FR-5: User Preference Initialization (Cold Start)

During onboarding, users must select preferred categories.

- These categories must be converted into an initial “Interest Vector.”
- This vector becomes the starting point for personalized recommendations.

FR-6: Dynamic User Profile Updating

The system must track articles a user reads or clicks.

- Each interaction should subtly adjust the user’s Interest Vector.
- Personalization must evolve over time based on actual behavior, not static preferences.

13.3 AI Vectorization Requirements

FR-7: Text Embedding Generation

All article summaries and user preference text must be converted into high-quality embeddings.

- Must ensure consistent vector representation for comparison.

FR-8: Efficient Vector Storage

Vector embeddings should be stored in a vector-friendly structure (e.g., DB with embedding support or serialized format).

- Retrieval should be fast to avoid latency during similarity calculations.

13.4 Recommendation Logic Requirements

FR-9: Personalized Feed Generation

Feed must be generated by computing cosine similarity between user vectors and article vectors.

- Only articles with similarity score greater than a threshold value should be shown.
- Threshold must be adjustable for refinement.

FR-10: Real-time Recommendation Updates

Whenever a user reads or interacts with an article, the system must immediately recalculate and update their feed relevance.

- Ensures personalization stays accurate and useful.

13.5 Semantic Search Requirements

FR-11: Meaning-based Search

The system must convert user queries into vectors and return semantically relevant articles.

- Should not rely on keywords alone.
- Must rank results by similarity score.

FR-12: Unified Search Across Categories

Search must not be restricted by topic/category; results should be based purely on semantic meaning.

- Should be able to find tech news even if user has never selected "Technology" during onboarding.

13.6 User Interface Requirements

FR-13: Personalized Feed Page

UI must display:

8. Relevant articles filtered by similarity
9. Article title, summary, source, and timestamp
10. Clear distinction between new and previously viewed articles

FR-14: Article Interaction Tracking

When a user clicks an article:

- System must log this event
- Trigger user profile vector update

14. Non-Functional Requirements

NFR1 - Performance

- System must respond to feed loading operations in < 2 seconds for a typical user.
- Semantic search results should be returned quickly.

NFR2 – Scalability

- Must support additional news sources without major architectural changes.
- Vector Store should handle growth in article volume

NFR3 - Security

- Store hashed passwords (never plain text).
- Ensure secure session handling and access control.

NFR4 - Reliability

- Automatic retry for failed RSS fetch attempts.
- Minimal downtime during data refresh operations.

NFR5 - Usability

- Simple, clean UI optimized for readability and quick scanning of headlines.
- Search and feed interactions must be intuitive.

NFR6 - Maintainability

- Modular architecture enabling easy updates to the AI model, data pipeline, or frontend components.

15. Risks & Mitigation

Risk	Impact	Mitigation
RSS feed structure changes	System breaks or fetch fails	Implement fallback parsing rules and error alerts.
Large article volume slows vector generation	Performance bottlenecks	Preprocess in batches; use caching for frequently accessed article vectors.
Cold start recommendation accuracy is low	Poor initial user experience	Improve category selection UI; allow users to manually refine interests.
AI model performance limitations	Lower relevance accuracy	Allow future model upgrades without breaking architecture.
User data privacy concerns	Loss of trust	Store no sensitive personal info; follow best practices for authentication.
Semantic search slow for large vector stores	Bad UX	Use approximate nearest neighbor search or vector indexing.

16. Module wise allocation

Module Title	Allocated to
News Aggregation	Arvind Sivakumar
User Management	Sahil Gupta
AI Vectorizer	Shuddhasattwa Majumder
Relevance Decision Module	Shruthika Kunnath Rajan
Semantic Search	Charumathi C
User Interface	Sriraam Vairavan R

17. High Level Design

System Overview: The system collects news article from online sources, cleans, and processes the text, models user preferences, and recommends relevant content to users based on their interest.

High Level Components:

1. News Collection Module: Fetches articles from RSS, feeds or APIs.
2. Preprocessing Module: Cleans text and prepares it for modeling.
3. Embedding / Vector Generation Module: Converts text into numerical vectors.
4. User Profile & Interest Module: Stores and updates user preferences.
5. Recommendation Engine: Matches user interests to articles.
6. Semantic Search Module: Finds articles based on query meaning.
7. Storage Layer: Saves articles, user info, and interaction data.
8. API & UI Layer: Provides interfaces for frontend and backend communication.

18. Low Level Design

1. News Collection Module (Python)

This module is responsible for gathering news from multiple online sources.

Internal Workflow

- A Python scheduler triggers the fetching process at regular intervals.
- RSS feeds are read using feedparser, while API-based sources use requests.
- Each fetched item is parsed to extract useful fields such as title, summary, article URL, published time, source name, and optional category.
- The raw data is stored in MongoDB in a simple structure so that preprocessing and embedding can happen later.

Purpose: Ensure a continuous flow of fresh news into the system.

2. Preprocessing Module

This module prepares raw article text for AI embedding.

Internal Workflow

- HTML tags and formatting noise are removed using BeautifulSoup.

- Unnecessary characters, line breaks, and symbols are cleaned using regex.
- Text is normalized to lowercase for consistency.
- Very long summaries may be shortened to a manageable length.

The cleaned text is stored back in MongoDB under a processed collection.

Purpose: Convert messy raw text into clean, standardized input for the AI model.

3. Embedding Generation Module (Sentence-BERT)

This module transforms text into numerical representations.

Internal Workflow

- Loads a pretrained Sentence-BERT model from Hugging Face.
- For each processed article: The summary is converted into an embedding vector.
- For each user: Their selected categories (like "sports", "technology") are combined into a text string and embedded to form a user vector.

User Vector Updating

- When a user reads or interacts with an article:
 - Their interest vector is slightly adjusted toward that article's vector.
- This creates *dynamic personalization*, reflecting changing interests over time.

Purpose: Provide a mathematical foundation for similarity-based recommendations.

4. Vector Storage & Similarity Index (FAISS)

FAISS enables fast comparison of article vectors.

Internal Workflow

- All generated article vectors are stored inside a FAISS index.
- When new articles arrive, their vectors are added to the index.
- MongoDB stores:
 - The article's ID
 - Its FAISS index position
 - Metadata like title and summary

Purpose: Allow fast lookup of “most similar articles” without scanning each one.

5. Recommendation Engine

This is the core intelligence of the system.

Internal Workflow

1. Retrieve the user's current interest vector from MongoDB.
2. Use FAISS to find the closest article embeddings.
3. Apply simple filters:
 - Too old articles dropped
 - Duplicates avoided
4. Return the top recommendations to the user.

Purpose: Give each user a personalized news feed based on what they like and what they read.

6. Semantic Search Module

Supports meaning-based search instead of keyword matching.

Internal Workflow

1. User enters a search query in the frontend.
2. Backend converts this query into an embedding using Sentence-BERT.
3. Vector is passed to FAISS to retrieve similar articles.
4. Articles are sorted by relevance and returned.

Purpose: Deliver intelligent search results that understand context and meaning.

7. User Interaction Tracking

This module improves personalization over time.

Events Captured

- Article clicks
- Time spent on an article
- Search queries
- Articles scrolled and viewed

Internal Workflow

- Each interaction is sent from the React frontend to the backend API.
- Data is saved in MongoDB.
- The user's vector is updated to reflect their recent reading behavior.

Purpose: Continuously refine recommendations using real user activity.

8. Backend API Layer (FastAPI/Flask)

Acts as the communication bridge between frontend and backend logic.

Internal Workflow

- Receives requests from the frontend (login, fetch feed, search, track activity).
- Interacts with MongoDB and FAISS to process each request.
- Sends back structured JSON responses containing:
 - Recommended articles
 - Search results
 - Article metadata

Purpose: Provide a clean, modular interface for all system operations.

9. Frontend (React)

A simple, clean interface for users.

Internal Workflow

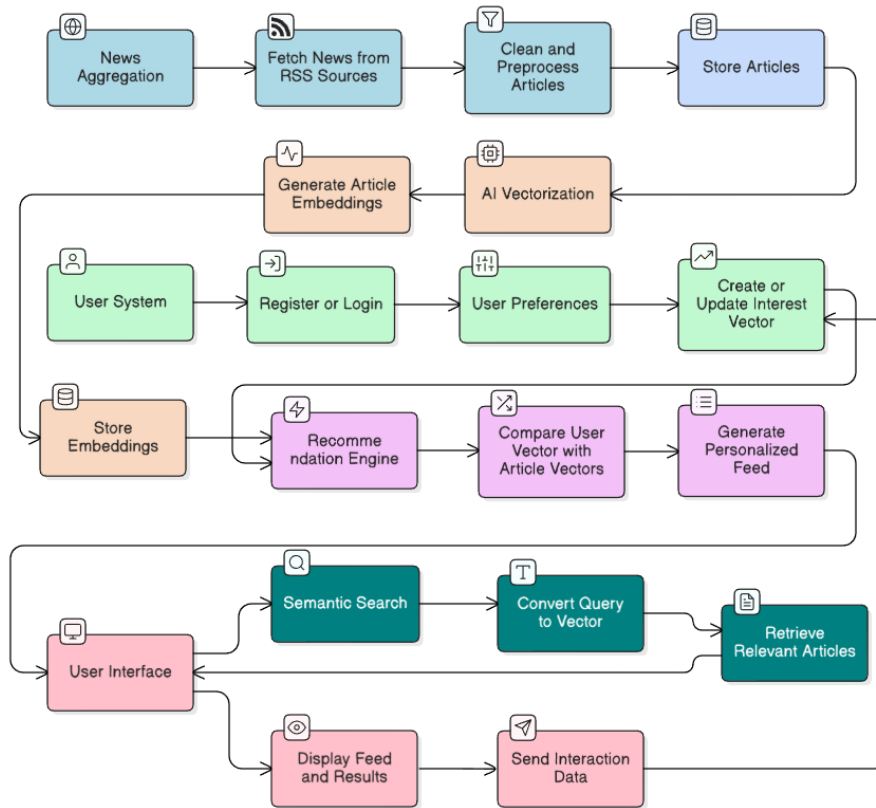
- Displays recommended news feed.
- Offers a search bar with semantic search support.
- Tracks user interactions (clicks, reading time) and sends them to the backend.
- Renders article details in a readable layout.

Purpose: Provide an intuitive and dynamic user experience.

19. Proposed Tech stack

Module	Technology / Tools	Purpose / Justification
News Aggregation & Preprocessing	Python	Used for fetching RSS feeds, scraping content where required, and preprocessing news articles.
	Requests / Feedparser	To retrieve news data from APIs and RSS feeds.
	BeautifulSoup / Regex	To clean HTML tags and preprocess article summaries.
Embedding Generation & Vectorization	Sentence-BERT (SBERT)	Used to generate semantic embeddings for news articles and user profiles.
	Hugging Face Transformers	To load and manage pre-trained SBERT models.
Vector Storage & Retrieval	FAISS / Weaviate	Used for efficient similarity search and local vector indexing during development.
	MongoDB (Optional Integration)	For storing article metadata, user profiles, and references to vector embeddings.
Relevance Decision Module	Python	Core language for similarity computation and recommendation logic.
	NumPy	Used for vector operations and cosine similarity calculations.
	Scikit-learn	Provides utilities for similarity computation and numerical processing.
Semantic Search Module	Python	Handles query processing and semantic matching logic.
	Sentence-BERT	Converts user search queries into embeddings.
	FAISS/Weaviate	Performs fast similarity search between query vectors and article vectors.
User Interface (Frontend)	JavaScript	Core language for client-side development.
	React (Tentative)	Used to build a responsive and component-based user interface for personalized feeds and search.
General Development Environment	Git / GitHub	Version control and collaboration among team members.
	Local Development Setup	System developed and tested in a local environment for academic purposes.

20. Proposed Architecture



21. Deliverables and Milestones

Phase	Duration	Milestones
Week 1	Requirements & Design	BRD completion, system architecture, module ownership allocation
Week 2		
Week 3		
Week 4		
Week 5		
Week 6		
Week 7		
Week 8		
Week 9		
Week 10		

22. Success Criteria

1. Users receive a personalized news feed with at least 70% relevance accuracy based on test feedback.
2. Search queries return semantically meaningful results consistently.
3. User Interest Vector updates result in noticeably improved recommendations over time.
4. System uptime > 95% during demonstrations.
5. End users (testers/classmates/faculty) report improved engagement vs generic news websites.
6. All modules function together without architecture-breaking issues.

23. Assumptions & Constraints

23.1 - Assumptions

1. Users will truthfully select categories during onboarding for initial personalization.
2. RSS feeds from selected news providers remain publicly accessible.
3. Sentence-BERT will run efficiently on available hardware.
4. Internet connectivity is stable for fetching external news sources.

23.2 - Constraints

1. Limited compute resources for real-time vector generation.
2. Fixed academic timeline.
3. Dependency on external RSS providers whose structure may change unexpectedly.
4. Must use open-source models/tools due to licensing and budget constraints.

Document Version: 2.2

Last Updated: December 14, 2025

Status: Draft (Awaiting Approval)

Version	Date	Type	Authors	Comments
0.1	01-Dec -2025	Baseline	Entire Team	Started working on the basic BRD
1.0	08-Dec-2025	Draft	Arvind Sivakumar	Initial formatted draft
1.1	13-Dec-2025	WIP	Arvind Sivakumar, Sahil Gupta	Added Architecture Diagram
1.2	13-Dec -2025	Baseline	Arvind Sivakumar	Added Exclusions
1.3	14-Dec-2025	Baseline	Shruthika	Project vision, Mission and objectives updated
1.4	14-Dec-2025	Baseline	Charumathi	Competitor Analysis and SWOT analysis

SS